

ELEVATEBOX · FULL STACK · FROM ZERO TO THE TOP 0.1%

The Operations Dashboard

The complete answer key & SDE-1 study guide — every first-principles question answered out loud, every concept the build forces, and exactly what you need to ship it.

One build · Five levels · 32 interview-grade answers · The full SDE-1 toolkit

Prepared as a study companion to the ElevateBox brief. Read each answer until you can say it without looking. That is the bar for SDE-1.

How to read this & the system in one picture

This document is your **defend-it-out-loud** companion. The brief tells you *what* to build; this tells you *why* *each piece exists* and gives you a model answer to every first-principles question an interviewer will ask. Each level has: the concepts it forges, the source→SOP mapping, and the Q&A.

The architecture, end to end

25 sources → **Cache** → **Widgets** → **Trigger rules** → **SOP Action Queue**. Nothing skips a step:

- **Sources (25):** 8 public (no key), 10 key-based, 7 scrapers.
- **Cache (TTL + stale-while-revalidate):** every API call goes through it — no exceptions. This is what keeps you under provider rate limits (e.g. Alpha Vantage's 25/day).
- **Widgets:** KPI cards, charts, tables, alerts. Each shows a visible **"Last Updated"** timestamp and answers one of three questions: *trigger an SOP now?* · *did we hit the SLA?* · *does the environment warrant an SOP refresh?*
- **Trigger rules:** `metric > threshold`. A breach drops a card into the Action Queue.
- **Action Queue:** a card with an assignee + an SLA clock. Idempotent — one real event makes exactly one card.
- **Refresh:** a cron / GitHub Actions job pulls data on a schedule — never on page load.

The rules that never bend: Every API call goes through the cache. No key ever enters git history (if one does, rotate it immediately). Every scraper respects `robots.txt` + rate limits. Swapping a provider is a single-file change. Every widget shows "Last Updated". Everything lives in version control from the first commit.

Level 0 — Foundations

GIT · GITHUB · THE WEB

Version control, and a real mental model of what happens between a click and a screen.

Concepts forged:

commit · branch · merge

repo · PR · review

.gitignore

client-server

DNS · IP · ports

HTTP methods & status codes

request lifecycle

HTTPS / TLS

Q1. HTTP is "stateless." Define that, and name the exact problem it creates the moment a user logs in.

Stateless means the server keeps *no memory of past requests*: each HTTP request must carry everything needed to be understood on its own, and the server treats request #2 as if it had never seen request #1. This is by design — it makes servers easy to scale horizontally (any server can handle any request) and easy to crash-recover.

The problem at login: after a user proves who they are with a password, the very next request has no inherent connection to that login — the server has already "forgotten" them. So nothing stops an unauthenticated request, and the user would have to re-send their password on every click. We bolt state back on top with a **session ID or JWT carried in a cookie**: the login response hands back a token, and the browser attaches it to every subsequent request so the server can re-identify the user without storing per-request memory in the protocol itself.

Q2. Walk through everything between hitting Enter on a URL and seeing the page. Where can it fail, where is it slow?

1. URL parse — browser splits scheme/host/path. **2. DNS resolution** — hostname → IP, checking browser cache → OS cache → resolver → root/TLD/authoritative servers. (*Fails: NXDOMAIN. Slow: cold cache, far resolver.*) **3. TCP handshake** — SYN/SYN-ACK/ACK to the IP on port 443. (*Slow: round-trips on high-latency links.*) **4. TLS handshake** — negotiate cipher, validate the server's certificate, derive session keys. (*Fails: expired/invalid cert. Slow: extra round-trips.*) **5. HTTP request** — **GET** + headers sent over the encrypted pipe. **6. Server work** — routing, auth, DB/cache hits, response built. (*Fails: 4xx/5xx. Slow: uncached DB queries.*) **7. Response** — status + headers + body travel back. **8. Render** — browser parses HTML → builds the DOM tree → CSSOM → render tree → layout (reflow) → paint → composite; it fetches sub-resources (JS/CSS/images) and runs JS along the way. (*Slow: render-blocking CSS/JS, large bundles, layout thrash.*)

Where latency hides: DNS (cold), the TLS round-trips, server-side DB calls, and render-blocking front-end assets — those are the four interview answers.

Q3. Real difference between TCP and UDP — and why does the web ride on TCP?

TCP is connection-oriented and reliable: it does a handshake, numbers every byte, retransmits anything lost, delivers bytes *in order*, and applies flow/congestion control. **UDP** is connectionless and fire-and-forget: no handshake, no ordering, no retransmission — just a thin wrapper that's faster and lower-overhead but lossy.

The web rides TCP because HTML, CSS, JS and JSON are useless if bytes arrive out of order or with gaps — a half-delivered script is broken, not "degraded." You need exact, ordered, complete delivery, which is exactly TCP's guarantee. UDP suits things that prefer freshness over completeness (live video, games, DNS queries). (Note: *HTTP/3 runs over QUIC, which is built on UDP but re-implements TCP-like reliability in user space to cut handshake round-trips.*)

Q4. 401 vs 403 vs 429 vs 500 — define each, and which layer emits it.

401 Unauthorized = "I don't know who you are" — missing/invalid/expired credentials. Emitted by the **authentication** layer (auth middleware). Fix: log in / refresh token.

403 Forbidden = "I know who you are, and you may not do this." Credentials are valid but you lack permission. Emitted by the **authorization** layer (RBAC/ABAC check). Re-authenticating won't help.

429 Too Many Requests = you've exceeded a rate limit. Emitted by the **gateway / rate-limiter**. Client should back off (honour **Retry-After**).

500 Internal Server Error = an unhandled error on the server — a bug or a downstream failure. Emitted by the **application** layer. It's *our* fault, not the client's; never use 500 for "you sent bad input" (that's 400).

Q5. Why does HTTPS matter for an internal tool "only the team uses"? What does TLS protect, and from whom?

"Internal" is not a security boundary — traffic still crosses Wi-Fi, switches, ISPs, VPN concentrators and cloud networks where anyone on-path can read or alter it. TLS gives you three things: **confidentiality** (encryption — an eavesdropper sees ciphertext, not your treasury numbers or session cookies), **integrity** (tamper-detection — nobody can silently inject or modify the response), and **authentication** (the certificate proves you're talking to the real server, not a man-in-the-middle). It protects you from anyone sitting between the browser and the server: a coworker on the same café Wi-Fi, a compromised router, a malicious ISP. Without it, a single sniffed session cookie hands an attacker a logged-in founder's session.

Q6. Your commit history is read before your code. What should it tell a senior that a single "final" commit never can?

A clean history tells the **story of your reasoning**, which a single squashed commit erases. Specifically: (1) **the order of decisions** — what you built first and why, so a reviewer can follow your thinking; (2) **small, isolated, reviewable units** — each commit does one thing, so a bug can be bisected to one change instead of buried in a 3,000-line blob; (3) **intent via messages** — *why* a change was made, not just what; (4) **safe revert/rollback** — you can undo one mistake without losing everything; (5) **collaboration trail** — branches and PRs show how feedback was incorporated. One "final" commit is a black box: unreviewable, un-bisectable, and it signals you don't work the way a team does.

Level 1 — The Frontend

HTML · DOM · JAVASCRIPT · 8 PUBLIC SOURCES

Get live data on screen from 8 keyless sources — and meet the browser for real.

Concepts forged:

semantic HTML

the DOM

accessibility

flexbox · grid · responsive

scope · closures

event loop

async/await · promises · fetch

reflow & repaint

React & components

state management

TypeScript

forms & validation

Source → widget → SOP trigger (A1–A8)

#	Source	Widget	SOP trigger
A1	CoinGecko (crypto)	KPI cards + 24h sparkline	Treasury Reserve Review · asset moves >10% intraday
A2	Frankfurter (FX)	30-day multi-currency line + rate KPI	Cross-border Invoicing · USD–client >2% WoW
A3	World Bank (macro)	Bar: HQ vs operating geographies	Pricing Review · inflation >2 SD from 10-yr mean
A4	Hacker News	Top-20 table + stories-per-domain bar	Inbound Press Response · client domain on front page
A5	WHO GH0 (health)	Heatmap of indicators by country	Compliance Audit · indicator worsens QoQ
A6	Open-Meteo (air quality)	Gauge per office + 48h forecast	WFH Advisory · AQI > 200
A7	RandomUser (mock HR)	HR table + avatar grid (seed data)	— seam to swap for a real HRIS later
A8	Reddit (sentiment)	Word cloud + complaints table	Competitive Intelligence · complaint spike >2× avg

Q1. What is the DOM, precisely — and why is it not the same as your HTML file?

The **DOM (Document Object Model)** is a live, in-memory **tree of objects** that the browser builds by parsing your HTML, and which represents the page *as it currently is*. Your HTML file is static text on disk — the initial blueprint, sent once. The DOM is the running structure built from it, and it diverges immediately: the parser fixes malformed markup and inserts missing tags (e.g. `<tbody>`), CSS and scripts attach, and then **JavaScript mutates it** — adding nodes, changing text, toggling classes. After `fetch()` renders a chart, the DOM contains nodes that appear nowhere in your HTML file. So: HTML = the recipe; DOM = the live dish you can keep editing. JS talks to the DOM, never to the file.

Q2. JS runs on one thread, yet your dashboard loads 8 APIs "at once." Explain the event loop — including where a fetch goes while it waits.

JavaScript has **one call stack** — it runs one thing at a time. But the work that *waits* (network, timers) is not done by that thread. When you call `fetch()`, JS hands the network request off to the **browser's Web APIs (host environment)** and returns immediately — the request is in flight "outside" the JS engine, managed by the OS/network stack. The thread is now free to fire the other 7 fetches and keep the UI responsive. When a response arrives, its callback (the promise's resolution) is placed on a **queue** — the microtask queue for promises, the macrotask queue for timers/events. The **event loop** is a simple rule: whenever the call stack is empty, take the next task from the queue and run it (microtasks drained fully before the next macrotask). So "8 at once" means: 8 requests are *in flight concurrently* in the host environment, and their callbacks run one-at-a-time on the single thread as they complete. Concurrency without parallelism.

Q3. What is a closure? Point to one in your own fetch-and-render code.

A **closure** is a function bundled together with references to the variables from the scope in which it was *defined* — so it keeps access to those variables even after that outer function has returned. In this dashboard, every async callback is a closure: `fetch(url).then(data ⇒ renderWidget(containerEl, data))` — the `.then` callback "closes over" `containerEl` and `url` from the surrounding function, and uses them later when the response arrives, long after the outer call returned. A caching wrapper is the cleanest example: `function makeFetcher(ttl){ const cache = {}; return (url) ⇒ cache[url] ?? ... }` — the returned fetcher closes over the private `cache` and `ttl`, giving each fetcher its own persistent, private state.

Q4. Reflow vs repaint — and why does mutating the DOM in a loop make a page stutter?

Reflow (layout) = the browser recomputes *geometry* — positions and sizes of elements — because something changed size/position (added a node, changed width, font, etc.). It's expensive because it can cascade through the whole tree. **Repaint** = the browser redraws *pixels* (colours, shadows, visibility) without changing layout — cheaper. A reflow always forces a repaint; a repaint doesn't force a reflow.

Why a loop stutters: if you mutate the DOM and then read a geometric property (`offsetHeight` , `getBoundingClientRect`) inside a loop, you force a **synchronous (forced) reflow** on every iteration — "layout thrashing" — instead of letting the browser batch changes into one reflow. 100 rows = up to 100 reflows on the main thread, blocking paint, so frames drop. Fix: batch DOM writes (build a `DocumentFragment` off-tree, insert once), separate reads from writes, or let the framework's virtual DOM batch for you.

Q5. A single number, a 30-day trend, a reading against a danger line — name the right chart for each and why the wrong one lies.

Single number → **KPI card** (big number, maybe a sparkline/delta). A chart for one value adds no information and invents a trend that isn't there.

30-day trend → **line chart**. Time is continuous and ordered, and a line encodes *direction and rate of change* — exactly what a trend is. A bar chart for a time series implies discrete, comparable categories and hides the slope; a pie chart for time is nonsense (it implies parts of a whole).

Reading against a danger line → **gauge or a line/bar with a threshold marker**. The point is "how close are we to the limit?", so the chart must show the value *relative to a reference*. A bare number ("AQI 210") without the 200 line forces the reader to remember the threshold — and a chart that omits the reference line hides the one fact that matters: whether we breached. The wrong chart "lies" because it encodes a relationship the data doesn't have (or hides the one it does).

Q6. `await` inside a loop vs firing all requests together — difference in wall-clock time, and why?

`await` in a loop is sequential: each iteration pauses until its request fully completes before the next even starts. 8 requests at ~300ms each $\approx$ **~2.4s** (they add up). **Firing all together** — `await Promise.all(urls.map(fetch))` — starts all 8 in flight at once and waits for the slowest, so total $\approx$ **~300ms** (the max, not the sum). The reason ties back to the event loop: the requests are independent and run concurrently in the host environment, so there's no reason to serialize them. Use sequential `await` only when request N+1 genuinely *depends* on N's result; otherwise parallelize. (*Caveat: respect rate limits — sometimes you intentionally throttle.*)

Level 2 — The Backend

NODE · EXPRESS · MERN · 10 KEY SOURCES

Ten key-based sources, and a server of your own: gateway → middleware → controller → cache/DB.

Concepts forged: Node event loop Express · routing middleware MVC / controllers REST design
validation MongoDB SQL vs NoSQL indexes & query plans transactions & ACID Redis · TTL
API gateway rate limiting retry/backoff idempotency secrets & .env

Source → auth → SOP trigger (B1–B10)

#	Source	Auth	SOP trigger
B1	Alpha Vantage	query key · 25/DAY	Investor Update · move >5%
B2	OpenWeatherMap	query key (appid)	Business Continuity · severe weather
B3	NewsAPI	header X-API-Key	Crisis Comms · client + negative tone
B4	FRED	query key	Pricing Review · CPI >0.3% MoM
B5	USAJOBS	Authorization-Key + UA email	Capture Management · new postings
B6	Clockify	X-API-Key	Capacity Reallocation · >90% for 3 wks
B7	Notion	Bearer + Notion-Version	SOP Refresh · unreviewed 6+ months
B8	Airtable	Bearer (PAT)	Escalate · onboarding incomplete >7 days
B9	Trello	key + token (OAuth1)	Escalate · card Blocked >3 days
B10	AQICN	query token	WFH Advisory · station-measured AQI >150

Q1. Node is single-threaded but serves thousands of concurrent users. How? And when does one CPU-heavy request bring it down?

Most server work is **I/O-bound** — waiting on DB, cache, network, disk — not CPU. Node runs your JS on one thread but uses a **non-blocking, event-driven model**: when a request hits I/O, Node hands it to the OS / libuv thread pool and immediately moves on to the next request instead of blocking. Thousands of users can be "in flight" because at any instant nearly all of them are *waiting*, and waiting costs the main thread nothing — it only does the small bursts of JS between I/O events. The event loop picks up each callback as its I/O completes.

It collapses on CPU-heavy work: a synchronous, long computation (image processing, crypto on a huge payload, JSON.parse of 100MB, an unbounded loop) *occupies the single thread* and the event loop cannot advance — every other user's request stalls behind it. Fixes: offload to **worker_threads** / a separate service / a queue, or chunk the work. The rule: never block the event loop.

Q2. What makes an API "RESTful"? Name the constraints, and why statelessness lets it scale.

REST is a set of architectural constraints: **client-server** separation; **statelessness** (each request self-contained); **cacheability** (responses say if they're cacheable); **uniform interface** (resources identified by URLs, manipulated via standard HTTP methods — **GET/POST/PUT/PATCH/DELETE** — with self-descriptive messages); **layered system** (client can't tell if it's talking to the origin or a proxy/gateway/cache); and optionally **code-on-demand**. In practice: resource-oriented URLs (**/clients/43**), correct verbs, correct status codes, no server-side per-client session state in the API itself.

Statelessness → **scale** because if no request depends on server-side memory of a previous one, *any* server instance can handle *any* request. That means you can put a load balancer in front and add/remove identical boxes freely (horizontal scaling) without sticky sessions or shared in-memory state. State lives in the client's token and in shared stores (DB/Redis), not in the app server — so the app server is disposable and replicable.

Q3. Trace one request through your backend, naming every layer and its single job.

1. Client — the browser sends the HTTP request. **2. API Gateway** — guards the door: rate-limiting (who, how often) and coarse auth; the one place these live so they aren't copy-pasted per route. **3. Middleware** — cross-cutting per-request work: parse/validate the body, authenticate (verify session/JWT), log, set CORS headers; rejects bad input early (400/401). **4. Controller** — the business logic: decides *what* to do, orchestrates the cache and DB, applies authorization (does this user own this record?). **5. Cache (Redis, TTL)** — checked first; a hit spares the DB. **6. Database (Mongo/SQL)** — the source of truth on a cache miss; an index makes the hot read fast. **7. Response** — serialize to JSON + the right status code, back through the layers to the client. Each layer has exactly one job; that separation is what makes it testable and changeable.

Q4. What is a database index actually doing under the hood — and a case where adding one makes things worse?

An index is a **separate sorted data structure** (usually a B-tree) that maps column values → row locations. Without it, a query scans every row ($O(n)$ full table scan). With it, the engine walks the B-tree in $O(\log n)$ to jump straight to matching rows — like a book's index versus reading every page. That's why reads on indexed columns are dramatically faster, and why you index the columns you filter/join/sort on.

When it hurts: every index must be *updated on every write* — so on a write-heavy table, indexes slow down **INSERT/UPDATE/DELETE** and cost extra storage. They also help nothing on low-cardinality columns (e.g. a boolean) where the planner may ignore them anyway, and a pile of redundant indexes confuses the query planner and bloats memory. Rule: index hot *reads*; don't over-index hot *writes*.

Q5. SQL vs NoSQL — what about the data's shape and access pattern decides it? One honest case for each.

Choose SQL (relational) when data is highly structured and relational, you need multi-row **ACID transactions**, strong consistency, and ad-hoc queries/joins across entities. *Honest case here:* the SOP Trigger Rules + Action Queue + audit log — you want referential integrity ("this action belongs to this rule"), transactional "create exactly one card", and to query "all breached SLAs last week". Relational fits.

Choose NoSQL (document, e.g. MongoDB) when data is document-shaped, schema varies between records, you read/write whole documents by key, and you favour horizontal scale and flexible schema over joins. *Honest case here:* caching the raw, differently-shaped JSON blobs from 25 heterogeneous providers — each source's payload differs, you fetch and store the whole document, and there are no joins. A document store fits that naturally. The deciding questions are: **is the data relational?** and **do reads follow a known key, or arbitrary joins?**

Q6. Why must every call go through a cache here? Do the Alpha Vantage arithmetic — and say what the widget shows on call 26.

The arithmetic: Alpha Vantage allows **25 requests per day**. If a widget called the API directly on every page load, then 25 page views — or one auto-refreshing dashboard polling every few minutes — exhausts the entire daily budget before lunch. With, say, 10 users refreshing, you'd blow the limit in minutes. So a direct-call design is broken by definition.

The fix: a cache with a long TTL (hours) means we hit the real API only a handful of times a day, on a schedule, and every page load reads the cached value — unlimited reads, ~24 upstream calls/day, comfortably under 25. **On call 26** (the limit exceeded), the API returns an error / 429; because every call goes through the cache + our retry-backoff wrapper, the widget **serves the last cached value and shows it flagged as "stale" with its "Last Updated" timestamp** — graceful degradation, never a blank or a crash. That's the whole reason the rule "every API call goes through the cache" never bends.

Q7. Where does an API gateway sit, and which two jobs would otherwise be copy-pasted into every route?

The gateway sits **at the front door — between the client and your route handlers/services** — so every request passes through it first. The two jobs it centralizes are **(1) rate limiting / throttling** and **(2) authentication** (plus often TLS termination, request logging, and routing). Without a gateway, every single route would have to re-implement "is this caller within their request budget?" and "is this caller authenticated?" — duplicated, drift-prone, and easy to forget on one route (which becomes the security hole). Putting them in one place means one consistent policy and one place to change it.

Level 3 — Identity, Access & Responsible Data

AUTH · RBAC · 7 SCRAPERS

Secrets and different users: a founder sees treasury data, an analyst does not. And being a polite guest on servers with no API.

Concepts forged:

authenticationsessions vs JWTHttpOnly-Secure-SameSitebcrypt/argon2

OAuth2 & OIDCMFAauthorizationRBACABAC & scopesleast privilege · IDORXSS · CSRF · SQLi

CORSsecrets mgmtresponsible scrapingbackground jobs · cron

Don't hand-roll auth. You must understand the mechanics (below) precisely because that's why you should reach for a vetted, free tool in production: **Better Auth** (open-source MIT, self-hosted TS — email/password, sessions, social, magic links, 2FA, RBAC), or Auth.js / Passport.js, or hosted free tiers (Supabase / Firebase / Clerk). The tool owns the crypto; you must still know *why* a salted slow hash, *why* HttpOnly cookies, and *where* authorization runs.

Scraper → catch → SOP trigger (C1–C7)

#	Source	The catch	SOP trigger
C1	SEC EDGAR	UA must carry name+email; ≤10 req/s	Material Event Memo · new 8-K, 24h SLA
C2	HN "Who is hiring"	robots Crawl-delay: 30	Org Design Refresh · a client is hiring
C3	RemoteOK	/api JSON; first element is a legal notice	Market Salary Benchmark · quarterly
C4	Wikipedia infobox	UA + contact, 1 req/s; prefer REST	Re-introduction Call · leadership change
C5	Yahoo Finance	JS-rendered; financials need Playwright (training only)	Market Event Memo · >5% intraday
C6	Wikipedia Pageviews	UA per Wikimedia policy	Reputation Audit · views >2× 30-day mean
C7	India MCA (data.gov.in)	download official CSV; don't scrape portal	KYC/Compliance Refresh · status change

Q1. Define authentication and authorization from first principles, and point to the exact moment each happens in your login flow.

Authentication (authN) = "who are you?" — a claim of identity plus a proof (password, token, OTP). It produces a *verified principal*. **Authorization (authZ) = "what may you do?"** — given a known principal, may this specific action on this specific resource proceed? AuthN happens **once, at the door**; authZ runs **on every request behind it**.

In the flow: authentication is the moment the user submits email+password (or "Login with Google"), the server verifies the bcrypt hash matches, and issues a session cookie/JWT — that single event. Authorization is every later moment: when they request `/api/treasury`, the server checks "does this principal's role (founder) permit treasury?" and when they request `/api/client/43`, it checks "does this principal own client 43?" Confusing the two — checking logged-in but not allowed — is the most common real-world security bug.

Q2. Why hash passwords instead of encrypting them? Separately: what does the salt defeat, what does the slowness defeat?

Hash, not encrypt, because encryption is *reversible* — it implies a key that, if stolen, recovers every password. Hashing is one-way: you store the hash, and at login you hash the input and compare. We never need the original back, so we should never be *able* to get it back. A breach then leaks hashes, not passwords.

The salt (a unique random value per password, stored alongside the hash) defeats **precomputation**: rainbow tables and "hash once, compare to millions of accounts" attacks. Because every password is salted differently, two users with the same password get different hashes, and an attacker must attack each account individually — no shared shortcut.

The slowness (bcrypt/argon2 are deliberately expensive, with a tunable cost factor) defeats **brute force**: it makes each guess cost real CPU/memory, so an attacker who can try billions of fast SHA-256 guesses per second is throttled to a relative trickle. Salt stops the shortcut; slowness raises the price of every single guess.

Q3. A session and a JWT both keep a user logged in. State the core trade-off in one sentence.

A session is stateful — the server stores it, so it's trivial to revoke instantly but needs a lookup/shared store that's harder to scale; a JWT is stateless — the server only verifies a signature, so it scales effortlessly but is hard to revoke before it expires. (Expanded: sessions keep state server-side — revoke = delete the row; JWTs carry signed claims in the token itself — no server store, but you can't un-issue one early without extra machinery like a blocklist or short expiry + refresh tokens.)

Q4. Explain CSRF: what makes it possible, and how does SameSite actually stop it?

What makes it possible: browsers *automatically attach cookies* to any request to a domain — including requests triggered by a *different* site. So if you're logged into the dashboard (auth cookie set) and you visit a malicious page, that page can silently submit a form or fire a request to `dashboard.com/transfer`, and the browser helpfully attaches your auth cookie. The server sees a valid, authenticated request and acts on it — even though *you* never intended it. The attack rides your ambient session.

How SameSite stops it: the `SameSite` cookie flag tells the browser *not to send the cookie on cross-site requests*. With `SameSite=Lax` (or `Strict`), when the malicious site triggers a request to your domain, the browser **omits the auth cookie** — so the forged request arrives unauthenticated and is rejected. It attacks the root cause (automatic cross-site cookie attachment). Defense-in-depth pairs it with CSRF tokens. (Note: `SameSite` doesn't help if the dangerous action is a `GET` with side effects — another reason GETs must be safe.)

Q5. What problem does CORS solve, and why is it NOT a security control protecting your server?

CORS (Cross-Origin Resource Sharing) relaxes the browser's **Same-Origin Policy**. By default a browser blocks JavaScript on `site-a.com` from *reading* the response of a fetch to `site-b.com`. CORS is the server's way of saying "it's OK for these specific other origins to read my responses" via **Access-Control-Allow-Origin** headers. It *enables* legitimate cross-origin calls (your frontend on one domain talking to your API on another).

Why it doesn't protect your server: CORS is enforced *by the browser, on the response, for the benefit of the user* — it stops a *browser* from *handing a cross-origin response to malicious JS*. It does nothing to stop a request from reaching your server: `curl`, Postman, a script, or any non-browser client ignores CORS entirely and your server still processes the request. So CORS is a browser-side convenience/safety feature, not server authorization. Real protection = authentication + authorization checks on the server, every request.

Q6. You verify the user is logged in, then serve `/api/client/43`. What check is missing, what's the bug class, what does it enable?

Missing check: object-level authorization — "does *this* authenticated user actually own / have permission to see client 43?" You checked `authN` (logged in) but not `authZ` *for this specific record*. **Bug class: IDOR — Insecure Direct Object Reference** (a.k.a. broken object-level authorization), the deadliest-because-quiet bug in the brief. **What it enables:** any logged-in user can simply change the URL to `/api/client/44`, `/45`, ... and read every other client's data — horizontal privilege escalation, a mass data breach by incrementing an integer. **Fix:** on every object access, verify ownership/permission server-side (`WHERE client.id = 43 AND client.owner = currentUser`), not just "is logged in." Never trust an ID from the client as proof of access.

Q7. RBAC vs ABAC — when does "roles alone" stop being enough?

RBAC assigns permissions to **roles** (founder, analyst, admin) and users to roles — coarse, simple, "who you are." It answers "may founders see treasury?" cleanly. **"Roles alone" breaks down when the decision depends on the relationship between the user and the specific resource, or on context** — e.g. "only the SOP's *author* may edit it," "only during work hours," "only for clients in your region." A role can't express "this user vs *that* record" without exploding into one role per object. **That's when you reach for ABAC** (Attribute-Based Access Control): a policy evaluates attributes of the user, the resource, and the context (`if user.dept == doc.dept and time is work-hours → allow`). It's fine-grained and dynamic. In practice: start with RBAC for broad gates, add ABAC/object-level checks (the IDOR fix) where permission depends on data and context.

Level 4 — System Design, Triggers & Shipping

THE LOOP · DEPLOY

Make it act: a number crosses a line, the right person gets the right SOP with a clock running. Then test, containerize, ship, watch.

Concepts forged:

event-driven triggers

idempotent actions

stateless services

load balancing · horizontal scaling

CAP & consistency

testing pyramid

CI/CD · GitHub Actions

Docker

cloud deploy

logging · metrics · tracing

health checks

observability

Q1. Define idempotency. Why must an action-creating trigger be idempotent, and how do you make one so?

Idempotency: performing the same operation once or many times produces the *same* result/state — repeats are no-ops, not duplicates. (`PUT x=5` is idempotent; "append a row" is not.)

Why the trigger must be: data refreshes run on a schedule and rules re-evaluate repeatedly; the same breach ("BTC -12%") will be seen on many cycles, retries happen after failures, and webhooks can fire twice. If creating an Action card weren't idempotent, one real event would spawn dozens of duplicate cards and the founder's queue becomes noise — exactly one real event must make exactly one card.

How: derive a stable **idempotency key** from the event (e.g. `ruleId + metric + breach-window/day` hash), and on creation do an **upsert** / **"insert if not exists"** keyed on it (a unique DB constraint, or check-then-create in a transaction). The second attempt finds the key already present and does nothing. Same idea protects retries and webhook re-deliveries.

Q2. Your dashboard runs on one box. Name three changes for 100× traffic — and what "stateless" has to do with all of them.

(1) Horizontal scaling behind a load balancer — run many identical app instances; the LB spreads requests across them. **(2) Externalize state** — move sessions/cache to shared Redis and data to a managed DB (with read replicas), so no instance holds unique in-memory state. **(3) Offload slow/async work to a queue + background workers** (scraping, trigger evaluation, card creation), and put a CDN in front of static assets.

Why "stateless" underpins all three: you can only add/remove/replace boxes freely if *any* request can go to *any* instance — which is only true if the app server keeps no per-user memory locally. The moment a server stores session state in its own RAM, you need sticky sessions, replacement boxes lose data, and the load balancer can't freely distribute. Statelessness (state pushed to shared stores + the client's token) is the precondition that makes load balancing, replication, and elastic scaling actually work.

Q3. What is the testing pyramid, and why many unit tests but few end-to-end?

The **testing pyramid** layers tests by scope: a wide base of **unit tests** (one function/component in isolation), a middle band of **integration tests** (modules together — controller + DB, route + middleware), and a thin top of **end-to-end tests** (the whole system through the UI, like a real user).

Many unit, few E2E because of cost vs. value: unit tests are fast (milliseconds), deterministic, and pinpoint *exactly* what broke — so you can have thousands and run them on every commit. E2E tests are slow, flaky (real network/timing/browser), expensive to maintain, and when one fails it tells you "something in the stack broke" without saying where. You want just enough E2E to prove the critical happy paths work together (login → data → trigger → card), and rely on the fast base for breadth. Inverting the pyramid (mostly E2E) gives a slow, brittle suite nobody trusts.

Q4. Should the threshold comparison run when fresh data lands, or when the page opens? Argue both ways.

On data-land (event-driven) — preferred here. Pro: triggers fire as soon as reality changes, independent of whether anyone is looking — a breach at 2am still creates a card and starts the SLA clock; evaluation happens once per refresh, not once per viewer; it's idempotent and centralized. Con: needs a background job/queue and slightly more infrastructure. This matches the brief ("an event arrives, a rule evaluates").

On page-open (lazy). Pro: simplest — no background worker, compute only when needed. Con (fatal for this product): if no one opens the dashboard, no SOP ever fires — the SLA clock that should have started at the breach instead starts whenever someone happens to look, so you can't honestly prove "we acted within 24h." It also re-computes per viewer and can produce inconsistent/duplicate firing.

Verdict: evaluate on fresh data; the page just *displays* the already-decided state. The action and the SLA clock must be tied to the event, not the audience.

Q5. What is a message queue actually for, and which part of this system most wants one?

A **message queue** (e.g. RabbitMQ, SQS, BullMQ on Redis) **decouples** a producer from a consumer: the producer drops a message and moves on; a worker picks it up later. It buys you **asynchrony** (don't block the request on slow work), **buffering / load-leveling** (smooth out spikes), **reliability/retries** (failed jobs redeliver, dead-letter on repeated failure), and **independent scaling** of producers vs. consumers.

Which part wants one most: the **refresh + trigger + action pipeline**. The 25 scrapers/API pulls are slow and fallible (perfect for queued background jobs that retry with backoff and respect rate limits), and the trigger→card creation should be a reliable, idempotent, at-least-once-with-dedup consumer — exactly the workload a queue is built for. You never want a 30-second SEC scrape inside a user's web request; you want it as a queued job.

Q6. An SLA is "24 hours." What must you record at the instant the trigger fires to later prove it was met?

At the firing instant, persist an immutable record containing at minimum: **(1) the trigger timestamp** (UTC — the clock's start, the most important field); **(2) the SLA duration / computed due-time** (start + 24h); **(3) the event identity** — which rule, which metric, the threshold and the actual value that breached, and the idempotency key; **(4) the assignee/owner** the card was routed to; and **(5) the SOP_ID**. Then, when the work happens, record the **completion/acknowledgement timestamp**. Met-or-breached is simply $\text{completed_at} \leq \text{fired_at} + 24\text{h}$. Store times in UTC and treat the firing record as append-only/audit-grade so the dashboard can *prove* — not just assert — the SLA was honoured. Without the start timestamp captured at fire-time, you can never audit it after the fact.

Reference — all 25 sources, endpoints & keys

Store every key in `.env`, load with dotenv, and never commit. Confirm `.env` is in `.gitignore` before the first commit.

Category A — Public, no key (8)

#	Source	Endpoint	Limit/note
A1	CoinGecko	api.coingecko.com/api/v3/simple/price?ids=bitcoin,ethereum&vs_currencies=usd	~30/min
A2	Frankfurter	api.frankfurter.dev/v1/latest?from=USD&to=EUR,GBP,INR	no documented quota
A3	World Bank	api.worldbank.org/v2/country/IND/indicator/NY.GDP.MKTP.CD?format=json	etiquette 1 req/s
A4	Hacker News	hacker-news.firebaseio.com/v0/topstories.json → /v0/item/{id}.json	cache aggressively
A5	WHO GHO	ghoapi.azureedge.net/api/NCDMORT3070	OData JSON
A6	Open-Meteo	air-quality-api.open-meteo.com/v1/air-quality?latitude=17.385&longitude=78.4867&hourly=pm2_5,pm10,ozone	none
A7	RandomUser	randomuser.me/api/?results=50&nat=us,in,gb	none
A8	Reddit	reddit.com/r/Entrepreneur/top.json?limit=25&t=day	set a descriptive User-Agent

Category B — Free key, registration required (10)

#	Source	Auth + limit	.env var (suggested)
B1	Alpha Vantage	key in query · 25/DAY (binding)	<code>ALPHAVANTAGE_KEY</code>
B2	OpenWeatherMap	query appid · 60/min	<code>OPENWEATHER_KEY</code>
B3	NewsAPI	header X-API-Key · 100/day	<code>NEWSAPI_KEY</code>
B4	FRED	key in query	<code>FRED_KEY</code>
B5	USAJOBS	Authorization-Key + User-Agent=email	<code>USAJOBS_KEY</code> , <code>USAJOBS_EMAIL</code>
B6	Clockify	header X-API-Key · 30/hr	<code>CLOCKIFY_KEY</code> , <code>CLOCKIFY_WORKSPACE</code>
B7	Notion	Bearer + Notion-Version: 2022-06-28	<code>NOTION_TOKEN</code> , <code>NOTION_DB_ID</code>
B8	Airtable	Bearer (PAT) · 5/s	<code>AIRTABLE_PAT</code> , <code>AIRTABLE_BASE</code>
B9	Trello	key + token in query · 300/10s	<code>TRELLO_KEY</code> , <code>TRELLO_TOKEN</code>
B10	AQICN	token in query · 1000/s	<code>AQICN_TOKEN</code>

Category C — Scraping, no formal API (7)

#	Source	How / the catch
C1	SEC EDGAR	data.sec.gov/submissions/CIK[...].json · UA MUST carry your name+email · ≤10 req/s
C2	HN "Who is hiring"	find latest thread, read each comment via Firebase item API · Crawl-delay 30
C3	RemoteOK	remoteok.com/api (JSON; first element is a legal notice — respect it)
C4	Wikipedia infobox	en.wikipedia.org/api/rest_v1/page/summary/{title} · UA+contact · 1 req/s
C5	Yahoo Finance	finance.yahoo.com/quote/AAPL · JS-rendered, Playwright · training only

C6	Wikipedia Pageviews	wikimedia.org/api/rest_v1/metrics/pageviews/... · UA per policy
C7	India MCA	download official CSV per ROC from data.gov.in · do NOT scrape the portal (CAPTCHA)

The SDE-1 concept map — everything to be able to explain

Tick each only when you can *explain and use* it without looking it up. That, not framework names, is the SDE-1 bar.

Foundations & Git

- commit · branch · merge
- repo · PR · review
- .gitignore
- client–server
- DNS · IP · ports
- HTTP methods & status codes
- request lifecycle
- HTTPS / TLS
- code review & semantic commits

Frontend

- semantic HTML · the DOM · accessibility
- CSS flexbox · grid · responsive
- scope · closures
- events & the event loop
- async/await · promises · fetch
- reflow & repaint
- React & components · state
- TypeScript · forms & validation
- routing · code splitting · API integration

Backend

- Node.js & its event loop
- Express · routing · middleware
- MVC / controllers · REST design
- validation & serialization
- MongoDB · SQL vs NoSQL
- indexes & query plans
- transactions & ACID
- caching · Redis · TTL

- API gateway · rate limiting
- retry/backoff · idempotency · secrets & .env

Data

- SQL vs NoSQL · documents
- schemas & relationships · keys & normalization
- indexes & query optimization
- transactions & ACID
- caching · TTL · stale-while-revalidate
- connection pooling · migrations

Auth & security

- authentication · sessions vs JWT
- cookies: HttpOnly·Secure·SameSite
- password hashing (bcrypt/argon2)
- OAuth2 & OIDC · MFA
- authorization · RBAC · ABAC & scopes
- least privilege · IDOR
- XSS · CSRF · SQLi · CORS
- secrets management

System design & ops

- event-driven triggers · idempotent actions
- stateless services
- load balancing · horizontal scaling
- CAP & consistency
- testing pyramid · CI/CD · GitHub Actions
- Docker & containers · cloud deploy
- logging · metrics · tracing
- health checks · observability
- queues & background jobs · feature flags · rollbacks

To actually ship it — build checklist & what you must provide

What I need from you (env / accounts) to build the live code

The 8 public sources (A1–A8) and the scrapers need **no keys** — those work immediately. To wire the 10 key-based sources, register (all free, no card) and paste the values into a `.env` file. Here is the template — fill what you can; any you skip, the dashboard degrades gracefully for:

```
ALPHAVANTAGE_KEY= # alphavantage.co (25/day – the binding limit)
OPENWEATHER_KEY= # openweathermap.org
NEWSAPI_KEY= # newsapi.org
FRED_KEY= # fred.stlouisfed.org
USAJOBS_KEY= # developer.usajobs.gov
USAJOBS_EMAIL= # your email (sent as User-Agent)
CLOCKIFY_KEY= # Clockify › Profile › API
CLOCKIFY_WORKSPACE=
NOTION_TOKEN= # developers.notion.com
NOTION_DB_ID=
AIRTABLE_PAT= # airtable.com personal access token
AIRTABLE_BASE=
TRELLO_KEY= # developer.atlassian.com
TRELLO_TOKEN=
AQICN_TOKEN= # aqicn.org
SEC_EDGAR_UA= # "Your Name your@email.com" (required by SEC)
# Auth (Level 3) + deploy:
BETTER_AUTH_SECRET= # random 32+ char string
GOOGLE_CLIENT_ID= # console.cloud.google.com (optional social login)
GOOGLE_CLIENT_SECRET=
DATABASE_URL= # Mongo Atlas or Postgres connection string
REDIS_URL= # upstash.com free Redis (optional; SQLite works too)
```

The build order (each is a single-file-swappable, cached, committed step)

1. **Repo first:** README, Node `.gitignore` (ignores `.env` + `node_modules`), feature branches + PRs, small message-led commits.
2. **Hit each endpoint with curl/Postman** before coding, to learn the real response shape.
3. **Cache layer (mandatory):** SQLite or Redis. Wrap every fetch: cache-hit-if-fresh → else fetch+store+TTL. Stale-while-revalidate; show "Last Updated". Long TTL for tight limits (Alpha Vantage = hours).
4. **Secrets:** all keys in `.env`; confirm it's gitignored before commit #1; rotate immediately if ever leaked.
5. **One retry/backoff wrapper:** on 429/5xx wait-and-retry (1s,2s,4s capped); on 4xx fail fast; on final failure serve cached + flag "stale".
6. **Backend (Level 2):** Express routes → middleware → controllers (no logic in routes); gateway for rate-limit + auth; deliberate data model + index where reads are hot.
7. **Auth (Level 3):** Better Auth (or Auth.js); founder vs analyst roles; enforce role *and object-level (IDOR)* checks server-side on every protected route.
8. **Scrapers (Level 3):** one committed YAML each (User-Agent+email, robots note, rate, selectors) — a reviewer can pause one without touching code. Run as scheduled background jobs.
9. **Triggers (Level 4):** an "SOP Trigger Rules" table (`(metric, comparator, threshold) → (SOP_ID, assignee, SLA_hours)`); ≥15 of 25 widgets create an **idempotent** Action card on breach; one Action

Queue scannable in <60s.

10. **Ship:** tests across the pyramid; GitHub Actions CI; Dockerfile; deploy to Vercel/Railway/Render (keys as host env vars); health-check route + logging; 25 one-pagers in `/docs`.

Honest scope note: this PDF is your complete *knowledge* deliverable — every answer you must defend out loud, plus the full reference. The live 25-source coded app is a multi-week build that needs your API keys, a database, and deploy accounts. Send me the filled `.env` above (or just say "start with the 8 public sources, no keys") and I'll scaffold the repo, the cache layer, the retry wrapper, and the first live widgets next.